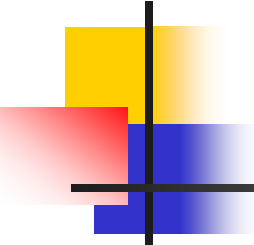


標準入力からの読み込みと ファイルの読み書き

プログラミング第2同演習B

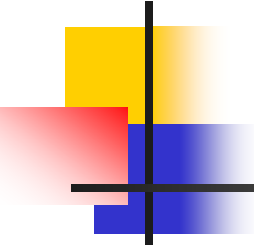
2026年6月19日

高田真吾



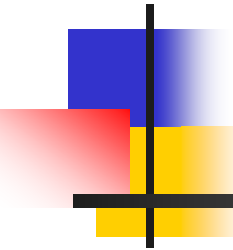
入出力の基本

- 標準入出力でも、ファイルでも、入出力の基本は：
 - ストリームを開く
 - 情報をストリームから読み込む(書き込む)
 - ストリームを閉じる
- ストリーム
 - データのシーケンスであり、入力の源または出力先を指す
 - 2種類
 - バイトストリーム
 - 文字ストリーム

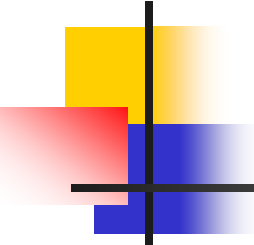


ストリームに関するクラスなど

- 一般的な(抽象)クラス: `InputStream`, `OutputStream`
- ファイルの入出力クラス: `FileInputStream`, `FileOutputStream`
- 標準入出力: `System.in`, `System.out` (← クラスではない!)
 - 標準入出力は, 開閉を気にしなくてもよい.
 - `in`と`out`は`System`クラスのクラス変数.
型についてはAPIドキュメントで調べましょう!

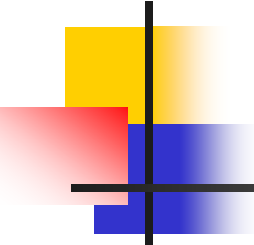


標準入力からの文字列の読み込み



文字列の読み込み

- シェルのように、文字列を一行ずつ入力したい場合がある.
- 例) Adderプログラム
 - 一行に整数をいくつか入力し、改行すると、その整数の総和が出力される. (数字と数字の間に空白があるとする.)
 - 上記が繰り返し実行される.
 - 「quit」を入力すると、終了する.
- 色々な方法がある.
 - ここでは Scanner クラスを使う方法を取り上げる.
 - 他にInputStreamReaderとBufferedReaderの組み合わせなどがある.



Adderの実行例

- Eclipse内で実行すると、下の方に「Console」というウィンドウが現れ、そこで入力する.

Please input numbers:

1 2 3

6

Please input numbers:

3 5 2 1

11

Please input numbers:

quit

*italic-underline*は
ユーザ入力

- 「quit」を入力した後、何も入力できなくなる.
 - 実行が終了する.



Adderクラス

```
import java.util.*;
```

```
public class Adder {
```

```
    private void processInput(String inString) {  入力処理するメソッド }
```

```
    public static void main(String[] args) {
```

```
        Adder app = new Adder();
```

```
        Scanner scan = new Scanner(System.in);
```

```
        while (true) {
```

```
            System.out.println("Please input numbers: ");  // プロンプトを表示
```

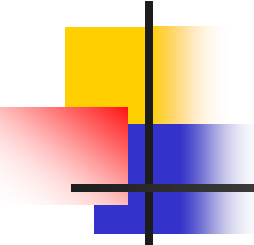
```
            String str = scan.nextLine();  // 次の行を読み込む
```

```
            app.processInput(str);  // 読み込んだ行を処理する
```

```
        }
```

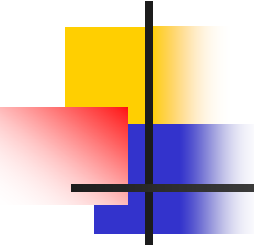
```
    }
```

```
}
```



Scannerクラス (1)

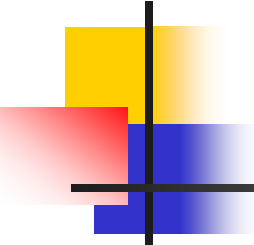
- 指定した源からテキストを取得し、様々な操作を可能とするクラス.
 - ここでは標準入力を例として取り上げているが、ファイルの入力などにも用いることができる.
- “源”は、Scannerのインスタンス生成時に指定.
 - `Scanner scan = new Scanner(System.in);`
 - 標準入力 (`System.in`) からテキストを取得するためのScannerインスタンスを生成している.



Scannerクラス (2)

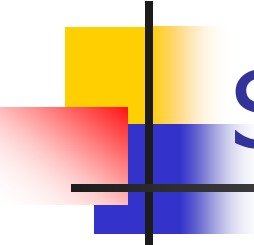
- Scannerクラスにはたくさんの
 - hasNextXX()メソッド
 - nextXX()メソッドを持っている.
- hasNextXX()メソッド
 - XX型として解釈できるトークンがあるか？
 - 例: hasNextInt()
→ int型として解釈できるトークンがあれば, true
- nextXX()メソッド
 - XX型として解釈できるトークンを返す.
 - 例: nextInt()
→ int型として解釈できるトークン(要するに整数)を返す.
- `String str = scan.nextLine();`
 - 次の行を文字列として読み込み, その参照を変数strに入れる.

APIドキュメントを見よう!



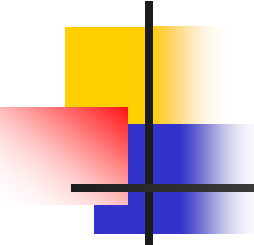
processInput()メソッド

- 主にやっていること
 - 空白で区切られた数字を取り出す.
 - 計算をしている.



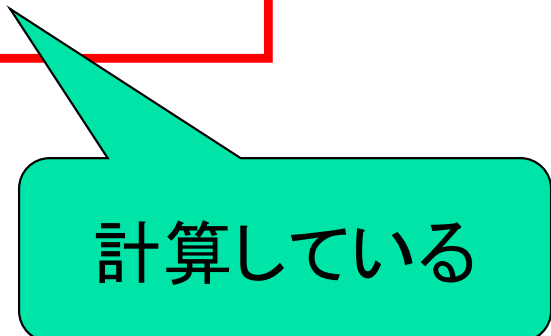
```
String[] inputs = inString.split("¥¥s+");
```

- 空白で区切られた数字を取り出す.
 - Stringクラスが提供する split メソッドを利用する.
String[] split(String regex)
この文字列を、指定された正規表現(regex)に一致する位置で分割します.
- ここでは, inStringが参照している文字列を,
空白文字(¥¥s)で区切り, 区切られた各文字列を
配列に入れて返す.
 - 空白文字が複数連続するかもしれないので, +をつける.
 - 「+」がない場合の入力
 - 「1(空白)2」は問題ない
 - 「1(空白)(空白)2」を入力すると何が起きるか試そう!



processInput()メソッド

```
private void processInput(String inString) {  
    String[] inputs = inString.split("¥¥s+");  
    int result = 0;  
    for (int i=0; i<inputs.length; i++) {  
        if (inputs[i].compareTo("quit") == 0) {  
            System.exit(0);  
        }  
        result += Integer.valueOf(inputs[i]);  
    }  
    System.out.println(result);  
}
```



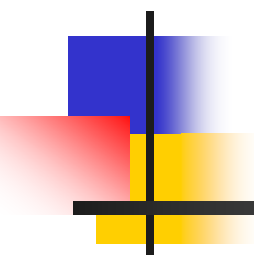
計算している

AdderXクラス (mainメソッドだけ)

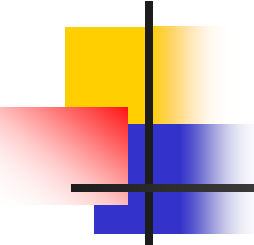
```
import java.util.Scanner;

public class AdderX {
    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);
        while (true) {
            System.out.println("Please input numbers: ");
            String str = scan.nextLine();
            String[] inputs = str.split("¥¥s+");
            int result = 0;
            for (int i=0; i<inputs.length; i++) {
                if (inputs[i].compareTo("quit") == 0) {
                    System.exit(0);
                }
                result += Integer.valueOf(inputs[i]);
            }
            System.out.println(result);
        }
    }
}
```

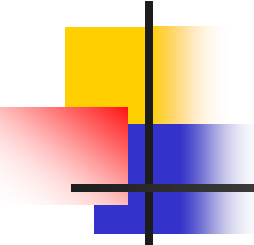


ファイルの読み書き



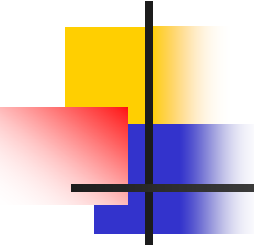
中心となるクラス

- ファイルの読み書きについてはいろいろなクラスが使える.
- 最も中心的なクラスとインタフェース
 - Files クラス(つづりに注意; 複数形)
 - Paths クラス(つづりに注意; 複数形)
 - Path インタフェース(つづりに注意; 単数形)
- ファイル読み込み時に使う(かもしれない)クラス
 - Scanner と FileReader と BufferedReader クラス
- ファイル書き出し時に使う(かもしれない)クラス
 - PrintWriter と FileWriter と BufferedWriter クラス
- 昔よく使ったが, 今あまり使わないクラス
 - File クラス(つづりに注意; 単数形)



Files クラス (つづりは複数形)

- ファイルやディレクトリなどを操作する static メソッドだけで構成されている.
- メソッド
 - `public static long copy(Path path, OutputStream out) throws IOException`
 - pathが示すファイルを, outにコピーする
 - `public static List<String> readAllLines(Path path, Charset cs) throws IOException`
 - pathが示すファイルからすべての行を読み込む.
 - 第2引数の「Charset cs」は, 基本的に「`Charset.defaultCharset()`」として, 現在の環境の文字セットを使うことでよい.
 - Java 8では, 第2引数がなくても大丈夫. (この場合, UTF-8文字セットを使用する.)
 - `public static BufferedWriter newBufferedWriter(Path path, Charset cs, OpenOption... options) throws IOException`
 - 新しいBufferedWriterのインスタンスを返す.



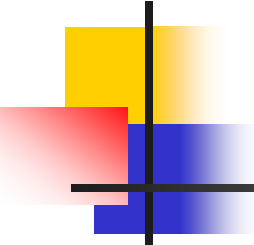
Paths クラスとPathインタフェース

■ Pathインタフェース

- ファイルを特定するために使用するオブジェクトであり、通常ファイルパスを表す.

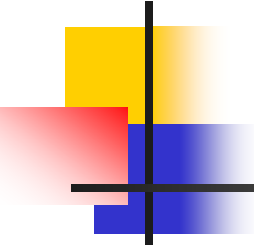
■ Pathsクラス

- パス文字列や URI を変換することによって Path を返す static メソッドのみで構成されている.
- `public static Path get(String first, String... more)`
 - ファイルパスに対応する文字列を引数として取り、それに対応する Path を返す.
 - `first` だけでパス文字列を指定することができれば、複数の文字列を指定して、それを結合したものをパス文字列にすることができる.



BufferedWriterクラス

- 文字をバッファリングすることによって、文字列などを効率良く文字型出力ストリームに書き込むためのクラス.
- メソッド
 - `public void write(String str) throws IOException`
 - 文字列を書き込む.
 - Writerクラスから継承.
 - `public void newLine() throws IOException`
 - 改行文字を書き込む.
 - `public void flush() throws IOException`
 - ストリームをフラッシュする.
 - フラッシュ: バッファにあるすべての文字をストリームに書き込むこと.
 - `public void close() throws IOException`
 - ストリームをフラッシュしてから, 閉じる.

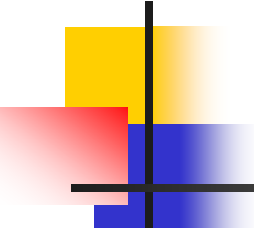


簡単な例 (1)

input.txtファイルの中身を
output.txtファイルにコピーする.

```
public class CopyFile {  
    public static void main(String[] args) {  
        try {  
            Files.copy(Paths.get("input.txt"),  
                        Paths.get("output.txt"));  
        } catch(IOException e) {  
            System.err.println("IOException " + e.toString());  
        }  
    }  
}
```

Files.copyメソッドはIOExceptionという
例外を投げる可能性があるため, try-catchを
使用しないといけない.



簡単な例 (2)

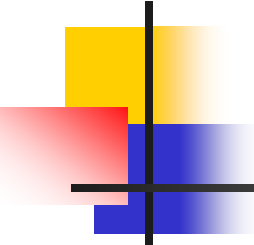
input.txtファイルの中身を
output.txtファイルにコピーする.

```
import java.io.*;
import java.nio.charset.*;
import java.nio.file.*;
import java.util.*;

public class CopyFile2 {
    public static void main(String[] args) {
        try {
            List<String> list = Files.readAllLines(Paths.get("input.txt"), Charset.defaultCharset());
            BufferedWriter output = Files.newBufferedWriter(Paths.get("output.txt"),
                                                            Charset.defaultCharset());

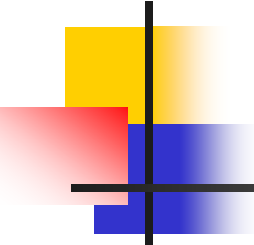
            for (Iterator<String> line = list.iterator(); line.hasNext(); ) {
                output.write(line.next());
                if (line.hasNext())
                    output.newLine();
            }
            output.close();
        } catch (IOException e) {
            System.err.println("IOException " + e.toString());
        }
    }
}
```

output.writeは改行しないので、
output.newLine()で改行文字を
書き込む。ただし、最後の行の
後に余計な改行をしないように、
if文がある。



Eclipseでの入力ファイルの置き場所

- 指定したパスが input.txt などの(ディレクトリを含まない)ファイル名の場合, プロジェクトのルートに置く.
- 例
 - Sampleというプロジェクト内に, CopyFileクラスを作成した場合:
 - CopyFile.javaの場所 → workspace/Sample/src の下
 - input.txtの置くべき場所 → workspace/Sample の下
 - output.txtができる場所 → workspace/Sampleの下
- 注意: 適宜 Refresh して, テキストファイルを更新した方がよい.



最後に...

- 入出力に関するクラスはたくさんある.
- APIドキュメントなどを見て, 習得するようにこころがけてください.